

La data e l'ora in Java

A partire da Java 8, la gestione di date e orari è stata notevolmente semplificata grazie al nuovo pacchetto **java.time**, che sostituisce le vecchie classi come **Date** e **Calendar** (presenti nel pacchetto `java.util`), ormai deprecate per molti usi.

Le classi principali introdotte sono:

- **LocalDate**: rappresenta una data (anno, mese, giorno) senza informazioni sull'ora o fuso orario.
- **LocalTime**: rappresenta un orario (ore, minuti, secondi) senza data né fuso.
- **LocalDateTime**: combina `LocalDate` e `LocalTime`, utile per rappresentare un momento specifico senza fuso orario.
- **ZonedDateTime**: include il fuso orario, ideale per applicazioni globali.
- **Instant**: rappresenta un punto nel tempo rispetto al timestamp Unix (1 gennaio 1970).

Operazioni comuni come ottenere la data corrente, formattare output, confrontare date o aggiungere giorni sono semplici e intuitive.

```
import java.time.LocalDateTime;
import java.time.LocalDate;
import java.time.LocalTime;

public class EsempioDataOra {
    public static void main(String [] args) {
        // Data e ora attuali
        LocalDateTime adesso = LocalDateTime.now();
        IO.println("Adesso: " + adesso);

        // Solo data corrente
        LocalDate oggi = LocalDate.now();
        IO.println("Oggi: " + oggi);

        // Solo orario corrente
        LocalTime ora = LocalTime.now();
        IO.println("Ora: " + ora);

        // Aggiungere giorni
        LocalDate domani = oggi.plusDays(1);
        IO.println("Domani: " + domani);

        // Confrontare date
        boolean futuro = oggi.isBefore(domani);
        IO.println("Oggi è prima di domani? " + futuro);
    }
}
```

L'uso di Date e Calendar (sua sottoclasse) in Java e il motivo della loro deprecazione

Date e i suoi problemi

La classe `Date` rappresentava un istante nel tempo, espresso come millisecondi dall'epoca Unix (1 gennaio 1970, UTC). Tuttavia, molti suoi metodi per accedere a componenti come anno, mese o giorno erano deprecati già dalla versione 1.1 perché:

- Anno: `getYear()` restituisce l'anno meno 1900 (es. 124 per il 2024).
- Mese: `getMonth()` usa un indice da 0 a 11 (gennaio = 0).
- Mancanza di internazionalizzazione: non supporta fusi orari o calendari non gregoriani.

```
Calendar cal = Calendar.getInstance();
cal.setTime(new Date()); // Imposta la data corrente
int anno = cal.get(Calendar.YEAR);
int mese = cal.get(Calendar.MONTH) + 1; // Attenzione: +1!
int giorno = cal.get(Calendar.DAY_OF_MONTH);
Date date = cal.getTime() // non restituisce l'ora,
                        // ma la data e l'ora
```

Calendar come soluzione parziale

Ad esempio metodo `getTime()` di `Calendar` restituisce un oggetto `Date`, utile per compatibilità, ma introduce confusione perché `Date` non gestisce bene fusi orari, mentre `Calendar` sì.

Perché sono state deprecate?

Le classi `Date` e `Calendar` sono considerate legacy perché:

- **Mutabili:** facilmente modificabili, poco sicure in contesti multithread.
- **API complessa e inconsistente:** metodi poco intuitivi, errori comuni (es. dimenticare l'indice del mese).
- **Gestione debole dei fusi orari e dell'ora legale.**
- **Mancanza di precisione:** solo millisecondi, non nanosecondi.

L'ora locale

In Java, l'ora locale viene gestita principalmente tramite la classe **`LocalTime`** del pacchetto `java.time`, introdotto in Java 8. Questa classe rappresenta un orario (ore, minuti, secondi, nanosecondi) senza fuso orario né data, ideale per scenari come orari di apertura, sveglie o eventi quotidiani.

```
//Ottenere l'ora corrente
LocalTime ora = LocalTime.now();
IO.println("Ora attuale: " + ora); // es. 14:35:22.123
//Creare un orario specifico
LocalTime apertura = LocalTime.of(9, 0); // 09:00
LocalTime pausa = LocalTime.of(13, 30, 45); // 13:30:45
//Operazioni comuni
LocalTime nuova = ora.plusHours(2); // aggiunge 2 ore
LocalTime futuro = ora.withMinute(59); // imposta i minuti a 59
boolean dopo = ora.isAfter(apertura); // confronta orari
//Formattazione
String formato = ora.format(DateTimeFormatter.ofPattern("HH:mm"));
IO.println("Ora formattata: " + formato); // es. 14:35
```

`java.util.Locale` non è correlato alla gestione di data e ora in Java, ma riguarda l'internazionalizzazione (i18n).

Serve per identificare una regione geografica, linguistica o culturale al fine di formattare correttamente date, numeri, monete e messaggi per utenti di diverse lingue.

Ad esempio, puoi usare `Locale` con `DateTimeFormatter` per mostrare una data in formato italiano:

```
LocalDate oggi = LocalDate.now();
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd MMMM yyyy", Locale.ITALIAN);
IO.println(oggi.format(formatter)); // es. "4 marzo 2026"
```

Senza **`Locale`**, la formattazione userebbe le impostazioni di default della JVM.

i18n è l'abbreviazione di internazionalizzazione (dove "18" indica le lettere tra la "i" iniziale e la "n" finale della parola *internationalization*). È il processo di progettare software, siti web o prodotti in modo che possano essere facilmente adattati a diverse lingue, culture e regioni senza modifiche al codice.

A cosa serve?

- Supportare più lingue e formati (date, valute, numeri).
- Gestire testi che si leggono da destra a sinistra (es. arabo).
- Separare il contenuto dal codice (es. tramite file di risorse).
- Preparare il prodotto per la localizzazione (l10n).

Esempio pratico:

In un'app Java, invece di scrivere: `IO.println("Benvenuto!");`

si usa una chiave: `IO.println(messages.getString("welcome"));`

che carica il testo corretto in base alla lingua (welcome=Welcome! in inglese, welcome=Benvenuto! in italiano).

DateTimeFormatter

La formattazione delle date in Java è fondamentale per presentare informazioni temporali in modo chiaro, leggibile e adatto al contesto culturale dell'utente. Senza una corretta formattazione, le date verrebbero visualizzate in formati standard come 2026-03-04T15:20:30, poco intuitivi per molti utenti.

`DateTimeFormatter` (introdotta in Java 8) è la classe principale del pacchetto `java.time.format` per formattare (da oggetto a stringa) e gestire il parsing (da stringa a oggetto) di date e orari:

```
LocalDate oggi = LocalDate.now();
DateTimeFormatter myPersonalFormat = DateTimeFormatter.ofPattern("dd MMMM yyyy", Locale.ITALIAN);
IO.println(oggi.format(myPersonalFormat)); // "4 marzo 2026"
```

Questo è essenziale per:

- Adattare il formato alla lingua (es. italiano, inglese).
- Conformarsi alle convenzioni locali (es. gg/mm/aaaa in Italia vs mm/dd/yyyy negli USA).
- Migliorare l'usabilità in applicazioni web, desktop o mobile.

Inoltre, la formattazione consente di esportare date in report, log o API in modo coerente e professionale.

Caratteristiche principali

- **Immutabile e thread-safe:** non può essere modificata dopo la creazione.
- Supporta formati personalizzati e predefiniti.
- Integrazione con `LocalDate`, `LocalTime`, `LocalDateTime`, `ZonedDateTime`, ecc.

Il metodo `DateTimeFormatter.ofPattern()` restituisce un oggetto di tipo `DateTimeFormatter`, che può così essere utilizzata per formattare e eseguire il parsing di oggetti data/ora come `LocalDate`, `LocalTime` e `LocalDateTime`.

Infatti, ad esempio, l'oggetto restituito può essere usato per:

- Convertire una data in stringa: `localDate.format(formatter)`
- Convertire una stringa in data: `LocalDate.parse(text, formatter)`

Perché è necessario il parsing della data in Java

Il parsing della data è essenziale perché le date spesso arrivano come stringhe (da input utente, file, API, database) e devono essere convertite in oggetti data/ora per poter eseguire operazioni come confronti, calcoli o formattazioni.

Esempio pratico

```
String dataStringa = "2026-03-04";
LocalDate data = LocalDate.parse(dataStringa); // Parsing automatico (ISO-8601)
IO.println(data.getDayOfWeek()); // martedì

// Parsing con formato personalizzato
dataStringa = "04/03/2026";
DateTimeFormatter myFormat = DateTimeFormatter.ofPattern("dd/MM/yyyy");
data = LocalDate.parse(dataStringa, myFormat);
IO.println(data.getDayOfYear()); // 63
```

Senza parsing, non puoi:

- Confrontare date (`isAfter`, `isBefore`)
- Aggiungere giorni (`plusDays`)
- Formattare in modo localizzato
- Validare correttamente l'input

Legenda dei pattern per DateTimeFormatter in Java

La classe DateTimeFormatter utilizza lettere specifiche nei pattern per rappresentare componenti di data e ora. Di seguito è riportata una tabella riassuntiva dei simboli più comuni:

Simbolo	Significato	Presentazione	Esempi
u	Anno (proleptico)	numero	2026, 04
y	Anno dell'era	numero	2026, 04
M	Mese dell'anno	numero	7, 07
L	Mese dell'anno	testo	Jul, July
d	Giorno del mese	numero	04, 15
D	Giorno dell'anno	numero	189
E	Giorno della settimana	testo	Tue, Tuesday
a	AM/PM	testo	AM, PM
H	Ora del giorno (0-23)	numero	0, 23
h	Ora del giorno (1-12)	numero	12, 1
m	Minuti	numero	30
s	Secondi	numero	55
S	Frazione di secondo	frazione	978 (millisecondi)
V	ID fuso orario	ID zona	America/Los_Angeles, Z
z	Nome fuso orario	nome zona	Pacific Standard Time, PST
X	Offset fuso orario (Z per zero)	offset	Z, -08, -08:30
x	Offset fuso orario	offset	+0000, -08:00
Z	Offset fuso orario	offset	+0000, -0800
'	Delimitatore testo	letterale	'test' → test
"	Singolo apice	letterale	" → '

Note:

- Il numero di lettere determina lo stile (es. MMM → Jul, MMMM → July).
- Per inserire testo letterale, racchiuderlo tra apici singoli: 'giorno'.
- Per inserire un apice nel testo, usarne due: ''.
- Il numero di lettere ha un limite di ampiezza (es. più di 4 y genera errore dal compilatore)

Anno proleptico si riferisce a un anno calcolato secondo le regole del calendario proleptico ISO, ovvero un sistema che applica le attuali regole del calendario gregoriano (inclusi i criteri per gli anni bisestili) anche a periodi storici precedenti la loro introduzione. Questo approccio è utilizzato per garantire coerenza e precisione nei calcoli temporali. Nel contesto di Java, il termine è frequentemente usato nel metodo `IsoChronology.isLeapYear(long prolepticYear)`, che verifica se un anno proleptico è bisestile.

Esempi di pattern e output

pattern	output	
dd/MM/yyyy	04/03/2026	Formato italiano: giorno, mese, anno
MMMM dd, yyyy	March 04, 2026	Mese per esteso, giorno e anno
E, MMM d	Wed, Mar 4	Giorno della settimana abbreviato, mese e giorno
yyyy-MM-dd HH:mm:ss	2026-03-04 14:30:25	Formato ISO con orario
dd-MM-yyyy'T'HH:mm:ss	04-03-2026T14:30:25	Separatore T tra data e ora
EEEE, dd MMMM yyyy	Wednesday, 04 March 2026	Giorno completo, mese completo, anno

Dalla Stringa alla Data:

Le classi DateFormat, SimpleDateFormat, ParseException

Le classi `java.text.DateFormat`, `java.text.SimpleDateFormat` e l'eccezione `java.text.ParseException` sono fondamentali per la gestione delle date in Java, soprattutto nelle versioni precedenti a Java 8 e oggi considerate legacy (deprecated).

DateFormat

Classe astratta che fornisce metodi per formattare e analizzare date in modo dipendente dal locale.

Fornisce metodi factory come `getDateInstance()`, `getTimeInstance()` e `getDateTimeInstance()` per ottenere istanze preconfigurate. Esempio:

```
DateFormat myDateFormat = DateFormat.getDateInstance(DateFormat.SHORT, Locale.ITALY);
String dataStr = myDateFormat.format(new Date()); // Es: "31/12/09"
```

SimpleDateFormat

Sottoclasse concreta di `DateFormat` che permette di definire pattern personalizzati per formattare e analizzare date. Esempio:

```
SimpleDateFormat mySimpleDateFormat = new SimpleDateFormat("dd/MM/yyyy HH:mm", Locale.ITALY);
Date data = mySimpleDateFormat.parse("25/12/2023 18:30");
String output = mySimpleDateFormat.format(data); // "25/12/2023 18:30"
```

Supporta i pattern supportati da `DateTimeFormatter` e riportati nella tabella sopra.

ParseException

Eccezione controllata lanciata quando il parsing di una stringa in data non riesce a causa di un formato non valido.

Casi d'errore comuni

- Formato non corrispondente: "25-12-2023" con pattern "dd/MM/yyyy"
- Locale non compatibile: nomi mesi in inglese con locale diverso
- Date non esistenti: 30/02/2023
- Orari non validi: 25:00 per l'ora

```
try {
    Date data = mySimpleDateFormat.parse("data errata");
} catch (ParseException err) {
    IO.println("Formato data non valido: " + err);
}
```

Nota importante: Queste classi sono non thread-safe e deprecated nell'uso diretto nelle applicazioni moderne. Si raccomanda l'uso delle API moderne (`java.time.*`) introdotte in Java 8 (es. `LocalDateTime`, `DateTimeFormatter`).

Formattazione e parsing (modern vs legacy)

Moderno (da preferire)

```
LocalDateTime now = LocalDateTime.now();
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
String formatted = now.format(formatter); // "04/03/2026 10:30"
LocalDateTime parsed = LocalDateTime.parse("04/03/2026 10:30", formatter);
```

Legacy (da evitare)

```
Date date = new Date();
SimpleDateFormat mySimpleDateFormat = new SimpleDateFormat("dd/MM/yyyy HH:mm");
// NON thread-safe: errore comune in ambienti multi-thread
String formatted = mySimpleDateFormat.format(date);
Date parsed = mySimpleDateFormat.parse("04/03/2026 10:30"); // lancia ParseException
```

Problema reale: SimpleDateFormat condiviso tra thread → risultati corrotti o ParseException imprevedibili.

Conversione da legacy a moderno e viceversa

```
// Da Date a LocalDateTime
Date legacyDate = new Date();
LocalDateTime modern = LocalDateTime.ofInstant(legacyDate.toInstant(), ZoneId.systemDefault());
// Da LocalDateTime a Date
Date back = Date.from(modern.atZone(ZoneId.systemDefault()).toInstant());
```

Pratica consigliata: Usa DateTimeFormatter come costante

```
public class DateTimeUtils {
    public static final DateTimeFormatter FORMATTER = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm");
}
// In questo modo si può usare FORMATTER in più thread senza rischi.
```

Strani risultati quando si passano parametri sbagliati o incoerenti

```
String var_data_input = "2025 63";
DateFormat format = new SimpleDateFormat("DD yyyy"); // il risultato non cambia anche con "dd yyyy"
Date obj_date = format.parse(var_data_input);
IO.println(obj_date); // stampa Dom Lug 17 00:00:00 CET 68 (Central European Time)
```

In questo caso sono stati considerati 2025 giorni dopo 1 gennaio 1963.

Questo problema deriva dal modo in cui i computer gestiscono le date e le ore, che è sostanzialmente all'interno di un unico numero.

Timestamp UNIX

Il timestamp Unix (o Epoch Time) è un sistema universale per rappresentare il tempo come numero di secondi trascorsi dalla mezzanotte UTC del 1° gennaio 1970, esclusi i secondi bisestili.

Caratteristiche principali

- **Punto di partenza:** 1970-01-01T00:00:00Z → valore 0
- **Valori negativi:** date precedenti il 1970 (es. 1969 → -31536000)
- **Indipendente dal fuso orario:** rappresenta sempre un istante unico
- **Facile da manipolare:** confronti, somme e differenze sono operazioni aritmetiche semplici

Ogni giorno inizia a un valore multiplo di 86400 secondi ($24h \times 60m \times 60s$). Esempio:

- ➔ 0 → 1° gennaio 1970, 00:00:00 UTC
- ➔ 86400 → 2 gennaio 1970, 00:00:00 UTC
- ➔ 1772640000 → 4 marzo 2026, 00:00:00 UTC
- ➔ 1772657326 → 4 marzo 2026, 20:48:46 (UTC)

I sottomultipli del secondo (millisecondi, microsecondi o nanosecondi) vengono gestiti principalmente in due modi, a seconda del sistema o del linguaggio di programmazione utilizzato:

- **Rappresentazione Decimale (Floating Point):** Il timestamp viene espresso come un numero decimale dove la parte intera sono i secondi e la parte frazionaria rappresenta i sottomultipli.
Esempio: 1772657326.500 (i .500 sono 500 millisecondi).
- **Rappresentazione Intera (Scaling):** Il timestamp viene "scalato" moltiplicando l'intero valore per la precisione desiderata. È il metodo più comune nei database e nelle API moderne per evitare errori di arrotondamento dei numeri decimali.

Millisecondi (ms): 13 cifre (es. JavaScript `Date.now()`).

Microsecondi (µs): 16 cifre (comune in Python o PHP).

Nanosecondi (ns): 19 cifre (standard nei sistemi Linux e Go).

Nota importante: Lo standard Unix Epoch originale definisce il timestamp strettamente in secondi. Quando vedi un timestamp molto più lungo di 10 cifre, quasi certamente è espresso in millisecondi o nanosecondi.

Lo standard ISO 8601

È lo standard internazionale per la rappresentazione di date, ore e intervalli temporali, progettato per eliminare ambiguità nei formati locali (es. dd/mm/yyyy vs mm/dd/yyyy).

Struttura principale

- Data: AAAA-MM-GG
- Esempio: 2025-03-04 (4 marzo 2025)
- Ora: hh:mm:ss (sistema 24h)
- Esempio: 14:30:00 (2:30 PM)
- Data + Ora: separati da T
- Esempio: 2025-03-04T14:30:00
- Fuso orario:
 - ♦ Z → UTC (es. T14:30:00Z)
 - ♦ +01:00 o -05:00 → offset rispetto a UTC

Caratteristiche chiave

- Ordinamento automatico: le date ISO si ordinano correttamente come stringhe.
- Formato esteso e base:
 - Esteso: 2025-03-04T14:30:00+01:00
 - Base: 20250304T143000+0100
- Giorno dell'anno: 2025-063 (3° marzo 2025)
- Settimana ISO: 2025-W10-3 (mercoledì della 10ª settimana)

Perché è importante

- Univoco: nessuna ambiguità tra Paesi.
- Macchina-leggibile: ideale per API, log, database.
- Supportato da Java: tramite `java.time` e `DateTimeFormatter.ISO_LOCAL_DATE_TIME`.